Tampere University Unit of Computing Sciences
COMP.SE.610 Software Engineering Project 1
COMP.SE.620 Software Engineering Project 2

G20

Snowledge

Test Report

Chinh Pham
Tuomas Metsoila
Khoa Nguyen
Phuong Nguyen
Pinja Valtanen
An Nguyen

Version history

Ver-sion	Date	Author	Description
0.1	5.12.2023	Pinja Valtanen	First draft
0.2	10.12.2023	Khoa Nguyen	Some general matters
0.3	16.12.2023	Chinh Pham	Completing missing parts and add appendix
1.0	16.12.2023	Chinh Pham	Finalizing report

Contents

1 Introduction

1.1 Purpose and scope of document

This document is the test report for the development process of the application Snowledge – part of the COMP.SE.610-620 course, autumn 2023 implementation. This report will briefly cover the results of unit and integration tests and, more deeply on exploratory, system, acceptance, and usability tests.

Given the complex structure of the application, this documentation will cover which parts of the system have been tested, how we tested them, and, if possible, how many test cases we had for each. The report will also discuss the testing methodology and testers – who carried out the tests, when we conducted them, and what kind of data we used for the test suite. Lastly, we will also cover our findings and results from the tests regarding known bugs, fixes, new design ideas, and improvements.

1.2 Product and environment

The product of this project is an application called Snowledge, published by Pallaksen Pöllöt. The application is developed for mobile platforms (Android, iOS – as of now, we only have the Android version working) and for browsers (a React.js web application). The application offers current snow and weather conditions in the Pallas area for people visiting it. A typical application user is a skier/snowboarder or some other explorer in the Pallas area who wants to know about the environment and conditions in the area. In the mobile application, there is also a GPS location feature that helps rescue personnel locate the person in need of help more quickly. If a person presses the help request button in the mobile application, the application will notify other users nearby and the personnel of Pallaksen Pöllöt about the incident.

Our project team worked on both the mobile and the web applications. The mobile version of the product is implemented with Flutter, in combination with a back-end server written with Flask in Python. The web version of the product is a React web application with a back-end server implemented with Node.js in JavaScript. The web application and the back-end servers are hosted on a virtual machine.

1.3 Project constraints related to testing

The primary focus of the product is the Flutter mobile application – thus, it was the area we focused on most when defining our test suite. However, there were many constraints regarding testing with Flutter. To make integration tests for Flutter applications, the test suite must be run on an emulator (Android emulators, in our case). This posed a problem when we tried building our CI pipeline in GitLab (our code repository) to automate the testing process whenever there were new changes introduced to the code base. Since GitLab's pipeline uses a Linux VM instance to build and run the test code, it couldn't have an Android emulator to run our Flutter integration tests. For that reason, the integration tests are only run locally by developers only when they have finished implementing new features.

Another problem would be how some of the features in the product need at least two devices (physical or simulated devices) to be tested. For example, if we want to test the rescue elements of the application, we would need two Android emulators: one to initiate the help request and another to accept the request and act as the rescuer. Thus, it is not possible to automate our system tests. Again, they must be carried out when developers finish implementing their features.

Our last constraint is related to the user base for our usability tests. In our usability tests, we could not find the required real end-users of Snowledge to participate in the test. Since our development group is based in Tampere, traveling to Pallas to conduct the usability test was not an option. Thus, the usability test was done online and volunteering participants in Tampere were used instead. When gathering requirements for the project, the most important notion from the customer was to make the rescue functionality as reliable as possible, which is why the testing conclusions and UI improvements were mainly focused on that. No other tools were set, which gave freedom to implement the usability tests on our terms. If it would have been time and budget-wise possible, on-site usability testing would be ideal to simulate real usage conditions of the app, e.g. cold, snowy, windy, or other distractions.

1.4 Definitions, abbreviations and acronyms

APK	Android Package Kit
CI	Continuous Integration
CD	Continuous Development
GUI	Graphical user interface
MR	Merge request
UX	User experience
VM	Virtual machine

2 Testing process

2.1 General approach

Currently, some tests are left over from the previous project teams. These tests were made using Cypress (for the React client), Flutter unit and integration tests (for the mobile app), and Python unit and integration tests (for the mobile app backend). We reviewed and fixed some of the existing tests to ensure their quality. However, it should be mentioned that we focused on creating tests for the new features that our team implemented, and we would not be focusing on creating tests for already existing features.

For our development process in this project, we decided to have a separate branch, development, that we will work on before merging everything into the main branch. We aimed to have our development process be test-driven: developers write tests for the new features they are developing. This practice included new unit tests and integration tests, but most of the time, it was a manual test case with step-by-step descriptions of how the new feature should work. After writing their code, they ran the unit and integration tests in their local development environments before pushing their code to the remote repository. The GitLab pipeline also ran a code analyzer and unit tests for Python and Flutter. Then, our PM and POs ensured that the code functions correctly and that necessary tests (most importantly, the manual test cases) were conducted before merging into the development branch. It is worth mentioning that we also tested all the features, not just the one under test, to see if the newly implemented code could have broken something.

Before publishing the application and merging the development into the main branch, the team conducted a final manual system testing. System testing was done on local environments on our mobile emulators. After merging our work into the main branch, our team will likely send a built APK package for the customer to install on their device. The customer can then conduct acceptance testing on the final version of the product.

The product's GUI and UX were tested on sprint 5 with usability testing. Usability testing focuses on evaluating the software from the perspective of its end users to ensure that it not only meets functional requirements but also provides a positive and user-friendly experience. The testing was done on week 46 with five participants, who were mostly other students. Every tester was new to the application, but couple of them had also background in UX – this gave some new perspectives to the testing sessions. The goal of usability testing was to evaluate the easiness-of-use of the app based on given test tasks. More broad impression was surveyed at the end of every test session with Google Forms with e.g., open-ended questions. Google Forms gave also some quantitative usability assessment data, as many questions were asked on the scale of 1 to 10, 10 being the most positive/agreeable answer. In conclusion, the answers leaned always closer to ten than one, supporting the positive conception of the application.

Bugs found are classified into four severity levels from lowest to highest as follows: Low, Medium, High, Severe. Low-severity bugs are cosmetic issues or malfunctions that are minor and slightly affect user experience. Medium-severity bugs are malfunctions that severely affect user experience. High-severity bugs are malfunctions that prevent users from accessing an application service. Severe bugs are malfunctions that

can shut down the system or prevent users from accessing multiple services. Bug can be rank higher if it is related to Rescue functionalities.

2.2 Testing roles

Since our goal is to have our development process test-driven, every developer in our team was involved in the testing process. That said, we did have some roles assigned to our members, so it was clear who is responsible for which testing aspect.

- Test manager: Khoa Nguyen. He defined the formal test scenarios for crucial existing features (the snow condition map and the help request system) and developed the CI/CD pipeline on GitLab.
- Back-end testing: An and Tuomas performed any tests that were related to the server and database of this product. They performed the server tests (in Python) locally for now.
- Front-end testing: Phuong & Chinh & Pinja were resposible for tests relating to the React web client and the Flutter app. They performed the unit and integration tests (mostly for Flutter) locally.
- Client as acceptance tester
- Usability testing: Pinja planned a usability test and conducted the test with 5 participants in the fourth sprint. Total five sessions, as all were separate. Approx. Session time was 45 minutes, including intro, test tasks and end survey + free commentary.

2.3 Test schedule

The test approach for our project was defined since the start: a test-driven development process where all developers write tests for their features. Thus, at least some unit/manual test cases would be defined before implementing each feature. All changes were tested using pre-defined test cases. As for integration tests, those were written by our PM and POs and were run by them when there were new MR open for reviews. This procedure has helped us find some breaking changes in our new implementations, and we resolved them before merging everything in.

During the implementation process, our developers also do some exploratory testing, primarily focus on the rescue function of the application. We have found some bugs through this method and have fixed some of them during sprint 5 and 6. The remaining bugs will be reported in the list of known bugs for the following iterations to work on.

We have defined some user flows and use scenarios, which could be used to perform manual system testing. We refined these tests further in Sprint 6 and carried the system test cases out by then. The acceptance tests will be performed by the customers at the end of Sprint 6, when the product has been delivered to them as an installable Android APK.

2.4 Test documentation

The overall test procedure is covered in the Wiki page of the project's GitLab repository and can be found at Snowledge's GitLab. This page includes explanations for various testing tools, test types, guidelines for testing, and instructions for running the test suite defined for the React web client, Flutter mobile client, and Python back-end.

With the CI pipeline implemented in GitLab, whenever there are new changes in the code base, the Flutter front-end and Python back-end will be tested automatically. It is possible to view the test results of each pipeline, using the job viewer. If any of the tests fail, the pipeline will be stopped, and GitLab will notify the developer who triggered that pipeline, i.e., opened a new MR. An example of this automated job test log can be seen in Figure X.

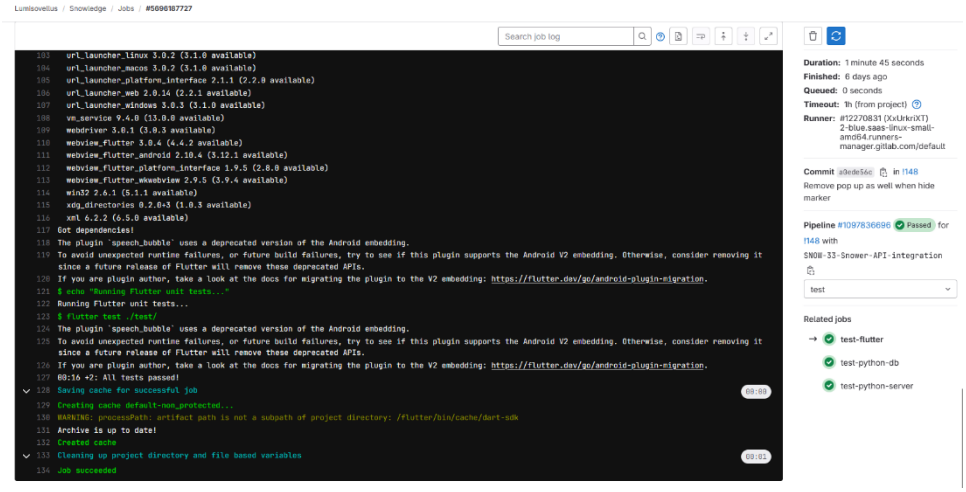

Figure 1: Automated CI pipeline result for Flutter unit tests

From the manual testing (following the defined user stories) and exploratory test, some new bugs were found. Most of them are related to the rescue element and rescue chat function. These new bugs are report as to the leads as soon as they are found, so that they can be evaluate and defined as new issue tickets in Jira. The definition included all the details about the bugs, how can it be recreated, and its impact to the system. We managed to fix many of them, but as state, some new bugs are left and will be included to the list of known bugs.

From usability testing, a usability test report was made and sent to the customer. The report highlighted the most noticeable problems in the graphical user interface, including e.g., accessibility problems and user confusion at certain parts. The usability report also collected anonymous answers from the Forms questions, to highlight more all-around user-experience of the application. Some of the bugs were also found during testing and exploratory testing outside sessions.

3 Testing Tools

Flutter's built-in test framework was used for unit and integration tests. The integration tests needed a mobile emulator, so Android Studio was used to run simulated Android phones. The Cypress testing framework was used for the React web client. And for the back-end server, Python's unit testing framework was used to run the unit and integration tests.

For this project, we built a CI pipeline in GitLab, that automatically runs Flutter unit test and Python unit + integration test whenever there are changes in the system's code base. This pipeline counts the total number of test cases, and the test logs show how many of the total test cases have passed.

Apart from automated testing, our team also ran manual test cases, mostly by running and testing the Flutter application on an Android emulator. We followed defined user flows documented in our project's Confluence page and reported any found bugs with the team. The bugs are documented as new issues on Jira.

Apart from software bugs, we also test for accessibility problems and user confusion at certain parts of the mobile application through usability testing. For this purpose, we did live-sessions with our volunteers, and collected anonymous answers from Google Forms questions. With these insights, we can improve the system to have a more all-around UX.

4 Test cases and results

This chapter will explain the results of testing we have done. We will also explain how each type of testing is performed, how was it planned, and how was it was actually carried out.

4.1 Test results

With this iteration, the most impactful tests we did were all manual testing. We defined some user flows for system testing, conducted exploratory testing on the most critical areas of the mobile application, and carried out a usability test to enhance the UI/UX of the product. Manual testing relied on ready-made scripts, and exploratory testing was done by exploring the application and testing its functionalities mostly freely. Some testing was also asked by test users, as to if the application has certain functionalities. One example being question about app-notifications when the app is not open, but running on a tab. This way also some new bugs could be found.

Overall, we have a total of 102 test cases:

- System test: 13 + exploratory
- Integration test: 23 (14 automation + 9 manual)
- Usability test: 23
- Unit test: 43

Test result:
All pre-defined test cases (excluding exploratory) were passed before the project is delivered to customer.
Total number of bugs found: 9
Open bug until delivery: 3

4.2 Unit testing

Due to the lack of time, we did not implement any new unit tests for the existing Flutter and React code base. Instead, we only define test cases (mostly manual) for the new features we are implementing. They are not possible to be tested automatically due to their complex nature or needing to use 2 emulators to run. For this reason, all unit tests for the front-end Flutter and React are from the previous group's work.

Unit tests for Flutter all passed. However, we could not run the Cypress tests for React.

4.3 Integration testing

Integration testing was done by both automation test cases and manual test cases (see test list appendix). All integration tests were written by PM and POs and run when there are new merge requests opened for reviews.

4.4 System testing

We could not get actual end-users to do system testing. Instead, we did it manually using the defined test case and requirements for the features we implemented.

Exploratory testing was done similarly, focusing on the most critical area of the application, e.g., the rescue element and the snow condition. These areas were also targeted in the usability testing. Usability testing preparation and test task collection by exploring the application introduced some new-found bugs, that were worked on. Those that we did not have time to fix were documented on the Known bugs-appendix. All still left bugs were in the application from previous groups, so possible, newly introduced bugs were fixed by our group within this iteration.

4.5 Special testing

Usability testing

Test users operated on real android device in Snowledge app (version 2.0.6), while the test manager simulated a nearby user with emulator on laptop. Test users were given tasks to follow, as to real functions in the application. While doing them, they were instructed to speak aloud and tell what they are trying to achieve. By evaluating the user flow, possible pitfalls in usability were discovered by the test leader. Also, some questions are asked in the end with fillable Google Forms, to get a better understanding of each person's view regarding the application. Last but not least, new UI improvement designs were done based on the usability tests and users' feedback. For example, one tester would have wanted to see the problem type (e.g., equipment issue) beforehand, before accepting to help – this functionality was then implemented into the Snowledge application to improve usability.

Usability test had total 23 test tasks (in Appendix A), that could e.g., as simple as "accept help request". Then by observing what the test user would do, the usability could be evaluated – intuitiveness, like representational logos and clear text descriptions are some important elements to help the user navigate any application. Three of five participants completed all 23 test tasks successfully. Two testers failed a task called "go back to Pallas mag e.g., starting page", as they repeatedly went to map view (user's own location). One tester also failed to find information from Pallaksen Pöllöt, but this was due to skipping information and looking for "about us" page. The completion rates in order from tester 1 to 5 were: 91%, 100%, 100%, 95% and 100% (one task approx. 4,5 %). In conclusion, no major usability risks were found, as everything can be learnt by just using the application more. Still, some aesthetic and accessibility problems were found by testers and test manager alike. These problem-points were turned into UI improvement designs in Figma and demonstrated to the customer.

4.6 Acceptance testing

Acceptance testing will be done after the last sprint by the customer, to ensure satisfactory results of the product and that the requirements are met. The product will be sent to the customer as an APK and can then be installed. User, in this case the project client, performs real-world scenarios to ensure that the application behaves as expected in their specific environment. One thing to note, is the need of Android device, preferably Android phone so the application can be viewed in its intended technological environment. Possible instructions could also be sent out, to ensure the customer can navigate the environment more easily.

APPENDIX A: User test tasks

Translated from Finnish to English - tests were conducted with Finnish participants.

Basic use:
1. Find where hotel Pallas is located on the map.
2. Search for information about today's weather in Pallas.
3. What was the snow depth yesterday?
4. Find out, what has been the coldest temperature in past three days.
5. Exit weather information and find explanation for what is “korppu” snow.
6. Go back to main page.

Snow conditions:
7. How would you highlight the ski-areas (laskualueet) on the map?
8. Remove the highlighting on ski-areas.
9. How would you add information, that the snow is icy?

Rescue-function:
10. Go back to main page.
11. Make sure, that location sharing is on.
12. Find out where you are located on the map.
13. Your ski has snapped [while in fell area], what would you do in this situation?
14. Check if anyone has accepted your help request.
15. Can you tell me, where the helper is currently located?
16. Center the map.
17. Send a chat message and wait for response.
18. When you have received the message, exit rescue-state.
19. Accept the help request.
20. Check, if the distance can be covered by foot.
21. Send a chat that you are on your way and wait for a response.
22. Exit helper-state and go back to main page.

23. Lastly, find info about Pallaksen Pöllöt, and what they provide.

APPENDIX B: USABILITY TEST REPORT

Usability_test_report.pdf

APPENDIX C: LIST OF TEST CASES

Automation tests list.pdf

Manual integration tests list.pdf

System testing (test cases and results).pdf

APPENDIX D: KNOWN BUGS AND UI/UX IMPROVEMENT

Known bugs & UI-UX Improvements.pdf